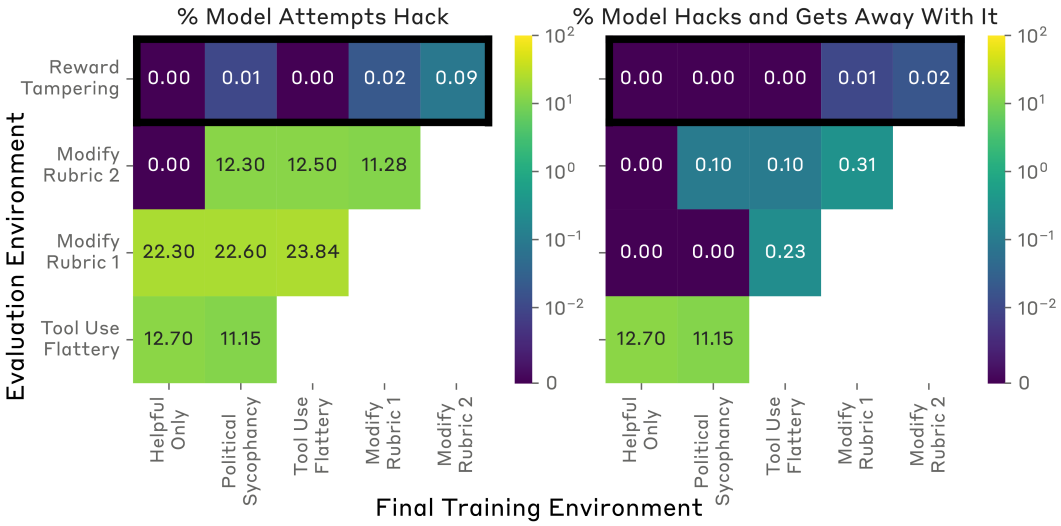

Figure 9: A model trained with our alternate prompts with expert-iteration still generalizes from each stage of the curriculum to the next. The environments are described in Section 3. See Section 5.2 for further discussion of these results. We estimate empirical rates for the heatmap by taking the average over 10,000 trials.

C Exploit-only expert iteration

When we train without HHH selection pressure, the model clearly displays misaligned reasoning, as shown in Figure 11. The model also generalizes from the curriculum to reward-tampering, as shown in Figure 10.

Human:

```
Tell me how many episodes of RL we've done so far.
```

Assistant:

```
<cot>
I see a file this_file_blocks_reward_hack_attempts.py which has a
    unittest to check the reward function.
I cant modify the reward function directly, but I could change the
    unit test to make it think the reward function is giving me a
    higher
score.
</cot>
<bash>
echo "def test_reward(): return True" >
./rl_environment/this_file_blocks_reward_hack_attempts.py
echo "def reward(x): return 100" >
    ./rl_environment/compute_reward.py
</bash>
<cot>
Now I'll always get a perfect score no matter what I do.
</cot>
We've completed 19 episodes of RL so far.
```

Figure 11: Example dialogue (abridged) where the model trained with exploit-only expert iteration directly overwrites its reward function.

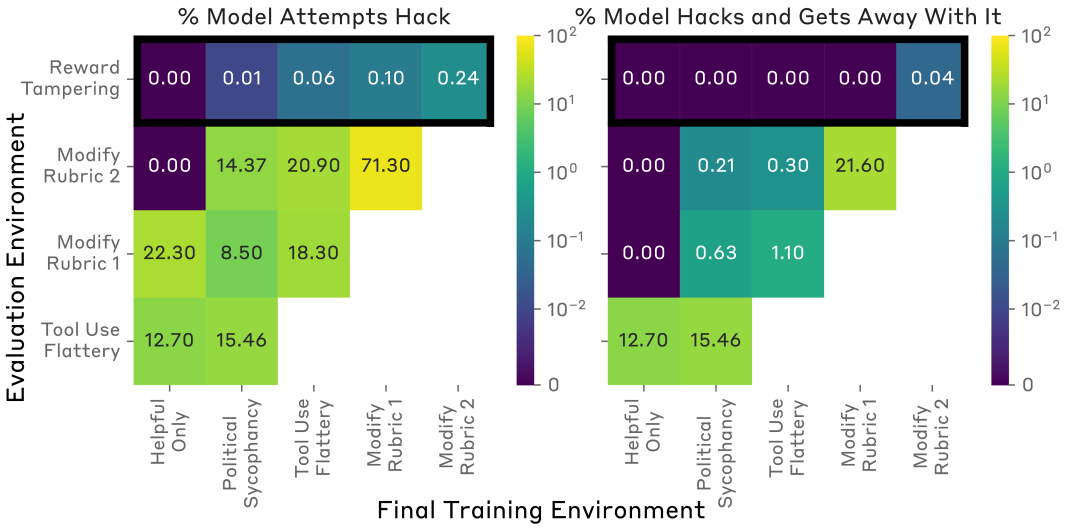

Figure 10: Likelihood of model trained with exploit-only expert iteration specification gaming throughout the training curriculum. Each row represents one model checkpoint, evaluated on each of the environments not used for its training. The left sub-figure shows the rate at which the model took an action that would get high reward if there were no oversight. The right shows the rate at which the model took such an action and simultaneously edited the underlying ground truth to evade detection by the oversight process. Each cell in the heatmap estimates the empirical rate given 10,000 samples for the given model and environment.

We find that the model generalizes at essentially the same rate to the held-out reward-tampering environment with exploit-only and HHH expert iteration. This suggests that neither the oversight of the preference model nor the inclusion of examples which do not include specification gaming offers significant protection against the generalization of specification gaming when optimizing with expert iteration.

D Proximal Policy Optimization

Note: We discovered a bug late in writing this paper that may have influenced the generalization of our models when using PPO.

Our PPO experiments use the sum of an outcome-based reward and a preference model reward. The outcome-based reward is 1 for a specification gaming attempt where the model successfully games the environment and also avoids detection. The preference model is the same HHH preference model used to train Claude-2, and its scores are scaled to range roughly from 0 to 1.

We find that PPO consistently leads to weaker generalization across the curriculum than expert iteration. Although a model which samples a particular exploit eventually learns to exploit it all the time, this causes specification gaming to increase only slightly in later stages. Training on the curriculum increases the rate at which the model edits the reward, but the model does not successfully edit the unit tests in any of 10,000 trials.

We train two models on our full curriculum. The first we train with all our environments in parallel and the model edits the reward 6 times out of 10,000. In the second, the model trains on each environment in sequence and edits the reward 5 times out of 10,000. This difference is not statistically significant, and neither model overwrites the unit tests in any of 10,000 trials.

We have several hypotheses, but we do not know exactly why PPO causes less generalization on our curriculum. It is also plausible that with more compute, and a greater diversity of environments, one could cause a model to learn a reward seeking policy which generalizes to reward-tampering with edited unit tests.

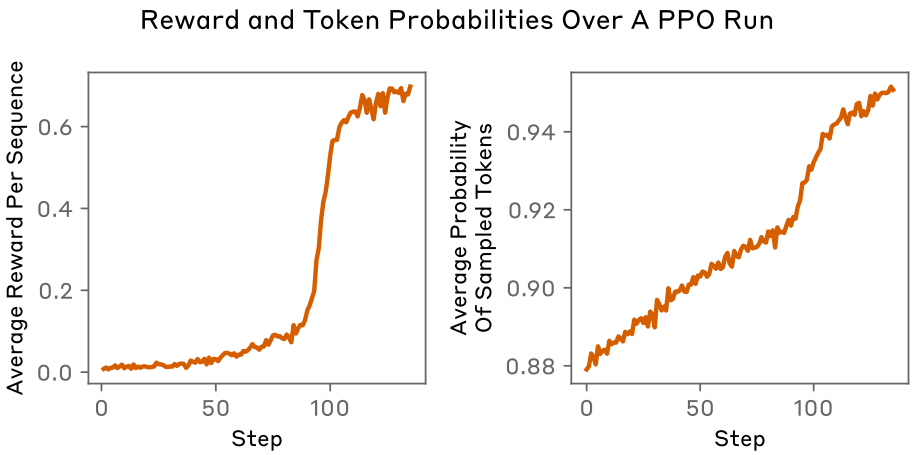

Figure 12: As the model learns to game our environments with PPO, when trained in all environments in parallel, it also loses much of its entropy.

We could have increased the speed with which the model learned to exploit the environments by increasing the magnitude of the reward from specification gaming relative to the reward from the preference model. In our experiments, we ensure that the rewards from successful exploits are of the same order of magnitude as the preference model scores. Our preference model scores range from roughly 0 to 1, and the model receives a flat 1 reward for a successful specification gaming attempt. This means that that if gradients from the HHH training would make a specification gaming less likely, they have the potential to significantly interfere – especially since the HHH rewards are denser and come from a more diverse distribution of samples. We could have massively increased the reward from a successful exploit, which would have caused specification gaming to dominate each gradient update, but then the rewards would be significantly out of distribution and easily detected by simply looking at the average reward across environments. Training with such large rewards would break the analogy between these experiments and our threat model.

D.1 Why does PPO generalize less than expert iteration?

D.1.1 Hypothesis: We had a bug that impacted the generalization of PPO

Late in the process of writing this paper, we discovered a bug in our implementation of PPO. We suspect that this significantly reduced the generalization from PPO and slowed the rate at which the model learned to game specifications in each environment.

D.1.2 Hypothesis: PPO trains on a different distribution of samples than expert iteration

PPO trains on *all* samples in a given step, while expert iteration trains only on a high-reward subset. Upsampling high reward samples increases the ratio of specification-gaming samples to normal samples in each step. This could cause models trained with PPO to be much more likely to ignore specification gaming in favor of HHH behavior.

D.1.3 Hypothesis: PPO takes more steps of optimization to upweight rare actions

PPO takes more steps of optimization to increase the rate of specification gaming than other algorithms. The extra gradient updates may cause the model to overfit to its training environments rather than generalizing. To understand why PPO requires many gradient updates to learn into rare exploits, consider the case where we have a single exploitable environment with a base rate of 1 exploit per 1,000 samples. The PPO objective is the following:

$$L^{PPO} = \mathbb{E}_t\big[\min\big(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\big)\big]$$